TD3 — toujours du CSS

Aujourd’hui : cibler plus précisément, positionner, organiser avec Flexbox.

1

CIBLER

Quel élément exactement ?

2

POSITIONNER

Où placer cet élément ?

3

ORGANISER

Comment le parent place ses enfants ?

LIEN AVEC TD2

La syntaxe ne change pas : sélecteur { propriété: valeur; }. Ce TD ajoute surtout des outils de sélection et de mise en page.

Préparation — téléchargez le ZIP du « TD3 — Cours »

SETUP

Après extraction, vous devez voir exactement ces 4 fichiers.

`A_advenced-selector.html`	Sélecteurs avancés
`B_position.html`	Position
`C_flex.html`	Flexbox
`styles.css`	Fichier CSS commun

MÉTHODE DU TD

1. Ouvrir le fichier HTML demandé dans le navigateur.
2. Ouvrir styles.css dans VS Code.
3. Modifier seulement la zone TUTO correspondante.
4. Ctrl+S → actualiser → comparer avec l'image attendue.

À FAIRE **Décompressez le ZIP. Gardez le navigateur et styles.css ouverts côte à côte.**

Comment travailler sur chaque étape

MÉTHODE

Chaque fois : regarder le HTML, modifier le CSS, observer le résultat.

1 REGARDER LE HTML

Quel élément est concerné ?

2 MODIFIER styles.css

Décommenter / écrire uniquement la règle indiquée.

3 ACTUALISER

Ctrl+S puis F5 / Ctrl+R.

4 COMPARER

Votre page doit ressembler à l’image « APRÈS ».

À FAIRE **Ne modifiez pas la partie « STYLE DE LA PAGE — NE PAS TOUCHER ».**

A — Sélecteurs avancés : d’abord lire le HTML

A • CIBLER

A, B, C, D sont seulement des noms de classes choisis pour l’exemple.

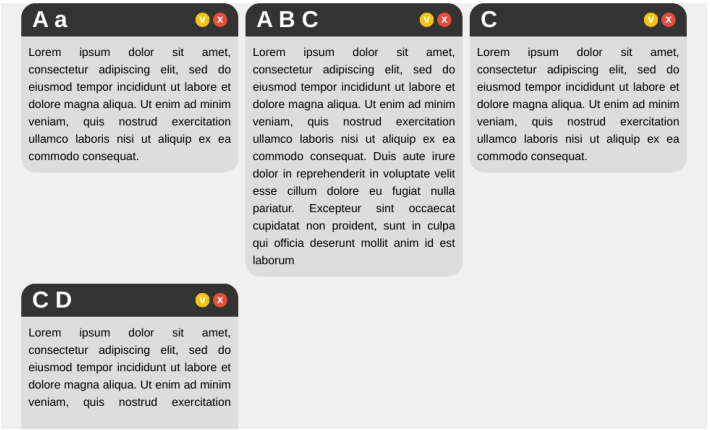
HTML RÉEL

```
<div class="window A" id="a">
<div class="window A B C">
<div class="window C">
<div class="window C D">
```

COMMENT LIRE CLASS="WINDOW A B C"

Un seul élément possède quatre classes : window, A, B et C. Les noms sont séparés par des espaces.

AVANT — aucun sélecteur du TUTO activé



À FAIRE **Ouvrez A_advenced-selector.html + styles.css. Dans « TUTO SELECTEURS », tout est encore commenté.**

A1 — Sélectionner une classe : .A

A • CIBLER

On veut modifier tous les éléments qui possèdent la classe A.

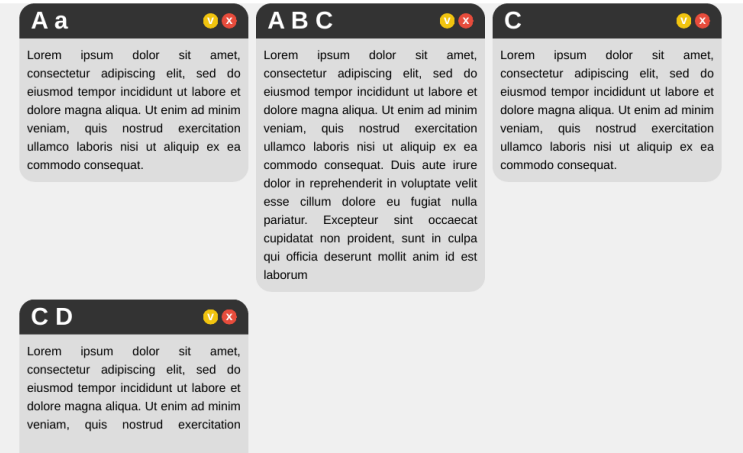
HTML CONCERNÉ

```
<div class="window A" id="a">
<div class="window A B C">
```

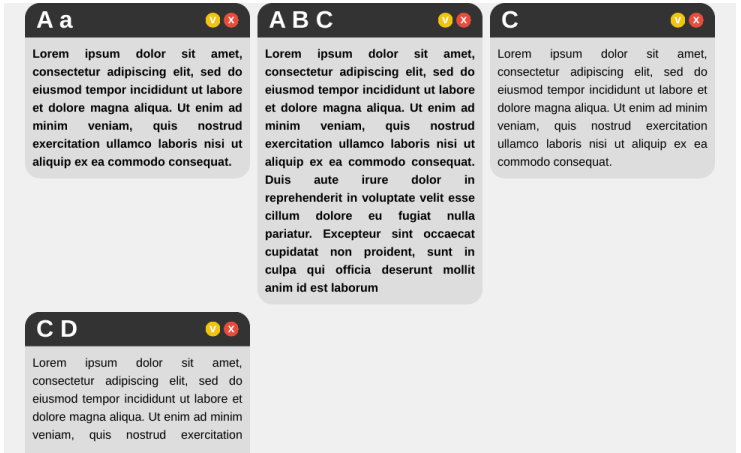
DANS STYLES.CSS — DÉCOMMENTEZ

```
.A {
    font-weight: bold;
}
```

AVANT



APRÈS



POURQUOI ?

Le point « . » signifie classe. Les deux premières fenêtres ont A dans leur liste de classes.

À FAIRE **Décommentez seulement .A, enregistrez, actualisez, puis re-commentez la règle avant l'étape suivante.**

A2 — Exiger deux classes sur le même élément : .A.B

A • CIBLER

Pas d’espace : A ET B doivent être sur le même élément.

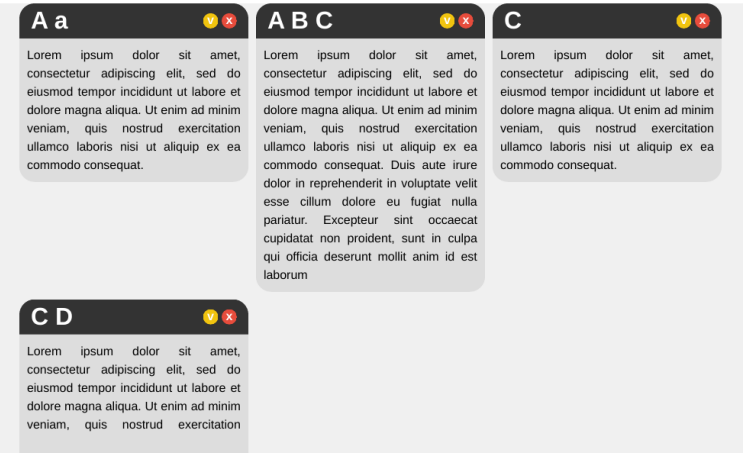
HTML CONCERNÉ

```
<div class="window A">
<div class="window A B C">
```

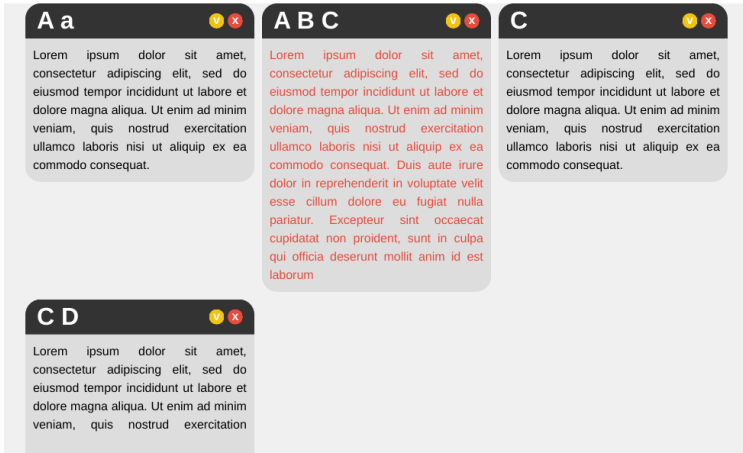
DÉCOMMENTEZ

```
.A.B {
    color: #e74c3c;
}
```

AVANT



APRÈS



RÉSULTAT

Seule la fenêtre A B C devient rouge : elle possède A et B en même temps.

À FAIRE **Décommentez .A.B, observez, puis re-commentez-la.**

.A.B et .A .B : l’espace change complètement le sens

Même symboles, deux relations HTML différentes.

SANS ESPACE

```
.A.B { ... }
```

HTML

```
<div class="A B">...</div>
```

SENS

Le MÊME élément possède A et B.

AVEC ESPACE

```
.A .B { ... }
```

HTML

```
<div class="A">
    <div class="B">...</div>
</div>
```

SENS

Un élément B se trouve À L’INTÉRIEUR d’un élément A.

À FAIRE **Ne tapez rien ici : retenez simplement « sans espace = même élément ; espace = descendant ».**

A3 — Lire un sélecteur très précis

A • CIBLER

On combine plusieurs conditions sur le même élément.

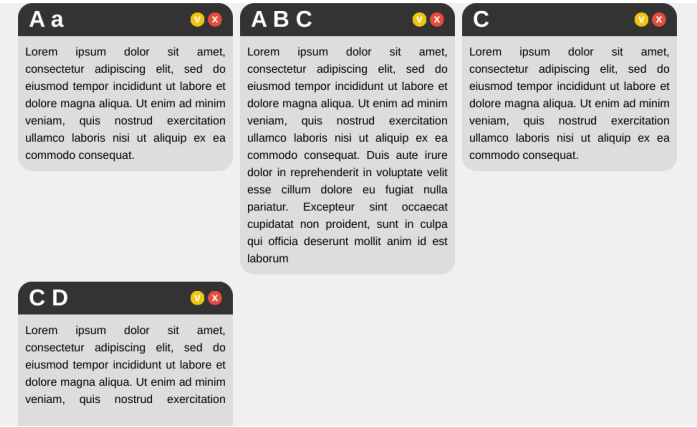
HTML

```
<div class="window A" id="a">
```

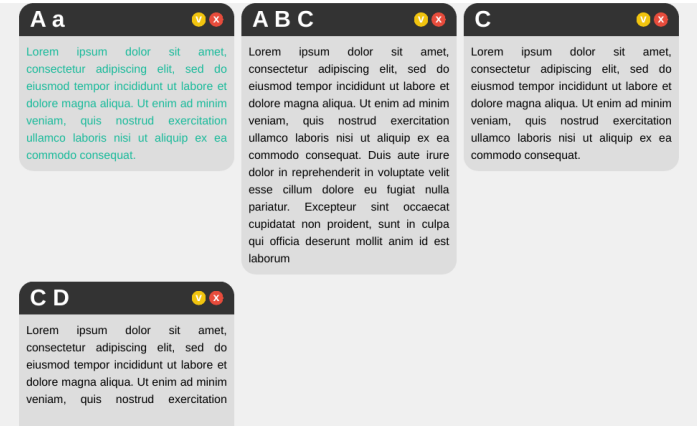
DÉCOMMENTEZ

```
div.window#a.A {
    color: #1abc9c;
}
```

AVANT



APRÈS



LIRE MORCEAU PAR MORCEAU

div = balise div | .window = classe window | #a = id a | .A = classe A

À FAIRE **Décommentez cette règle, observez la fenêtre « A a », puis re-commentez-la.**

A4 — Exclure avec :not()

On sélectionne C, mais pas les éléments qui possèdent aussi D.

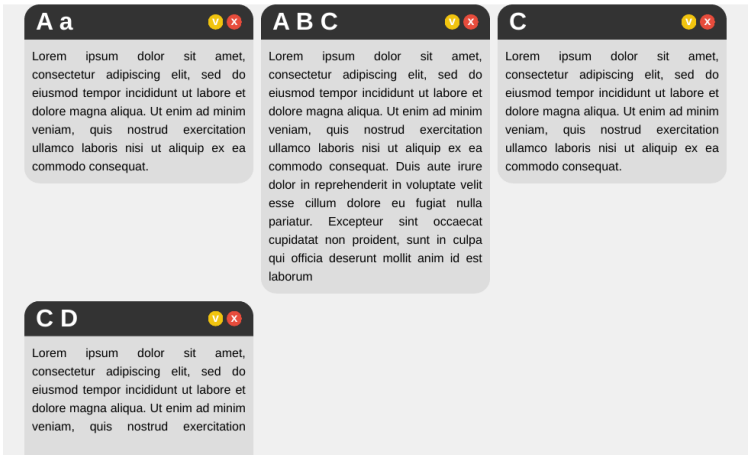
DÉCOMMENTEZ

```
.C:not(.D) {
    color: #9b59b6;
}
```

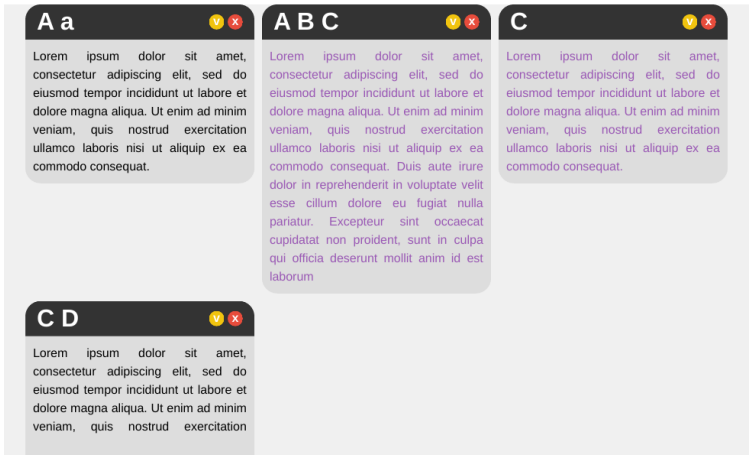
LECTURE

1. prendre les éléments de classe C
2. :not(.D) = exclure ceux qui ont aussi D

AVANT



APRÈS



À FAIRE **Décommentez :not(), observez A B C / C / C D, puis re-commentez.**

A5 — Sélectionner selon la position : :nth-child()

Testez une règle à la fois pour éviter de mélanger les effets.

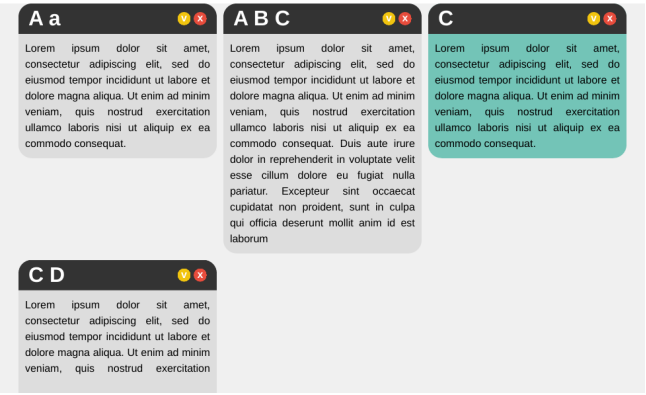
TEST 1 — DÉCOMMENTEZ

```
.window:nth-child(3) {
    background-color: #73c4b7;
}
```

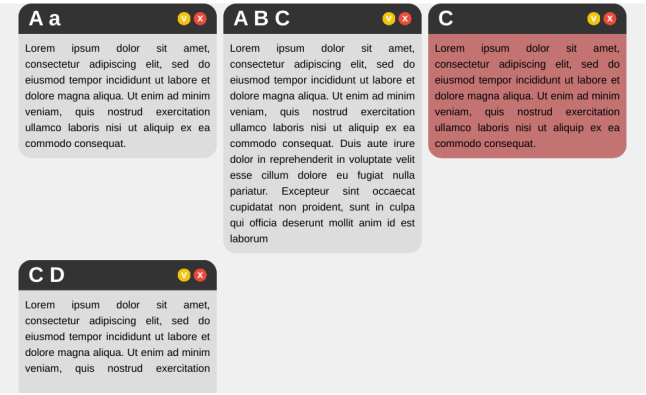
TEST 2 — À LA PLACE

```
.window:nth-child(3n) {
    background-color: #c47373;
}
```

RÉSULTAT — enfant n°3



RÉSULTAT — 3, 6, 9…



À RETENIR

3 = troisième enfant. 3n = chaque troisième enfant.

À FAIRE **Testez d’abord :nth-child(3), re-commentez, puis testez :nth-child(3n).**

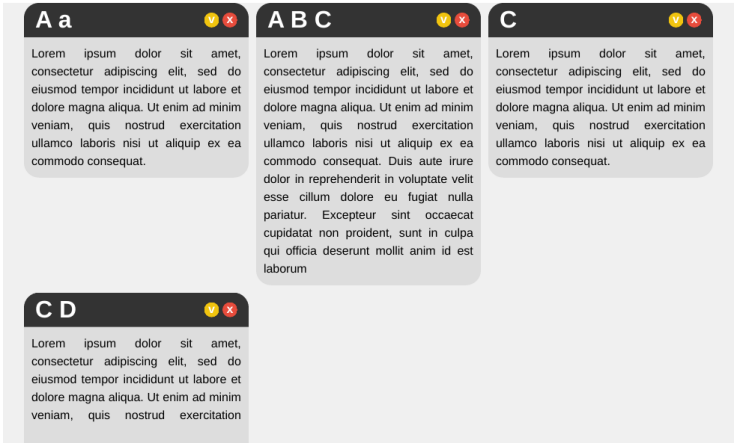
A6 — Réagir à un état : :hover

La règle s’applique seulement pendant le survol de la souris.

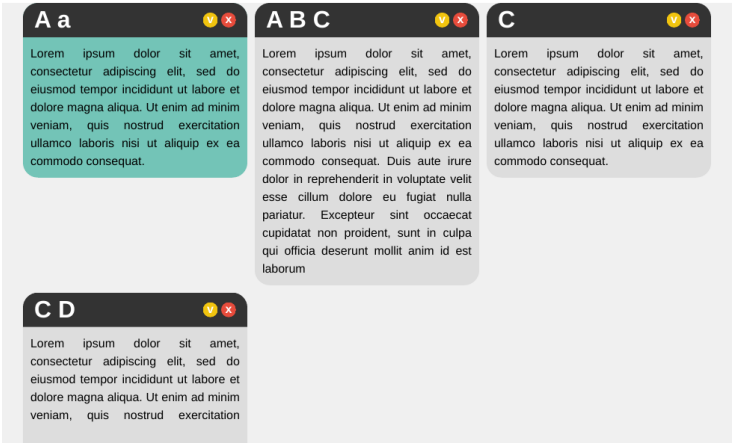
DÉCOMMENTEZ

```
.window:hover {
    background-color: #73c4b7;
}
```

AVANT



APRÈS



DIFFÉRENCE AVEC TD2

Une règle normale est permanente. :hover ajoute une condition liée à l’état de l’élément.

À FAIRE **Décommentez :hover et déplacez réellement la souris sur plusieurs fenêtres.**

B — Position : ouvrir B_position.html

Ici, les règles sont déjà écrites : on les lit et on teste leur effet.

HTML

```
<div class="window static">
<div class="window relative">
<div class="window sticky">
<div class="window absolute">
<div class="window fixed">
```

FLUX NORMAL

C’est l’ordre naturel dans lequel le navigateur place les éléments avant un positionnement spécial.

IMPORTANT

Contrairement à la partie A, il n’y a rien à décommenter au départ. Les blocs .static, .relative, .absolute, .fixed et .sticky sont déjà actifs.

RÉFÉRENCE — si tout était en position static



À FAIRE **Ouvrez B_position.html. Repérez dans styles.css les 5 blocs « TUTO POSITION ».**

B1 — static puis relative

Avec relative, l’élément part de sa position normale puis se décale.

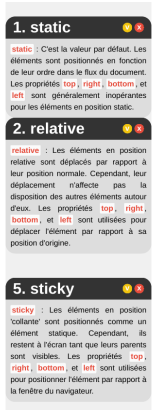
BLOC DÉJÀ ÉCRIT

```
.window.relative {
    position: relative;
    left: 50px;
    top: 50px;
}
```

TEST À FAIRE

Commentez temporairement left et top → actualisez.
Puis remettez left:50px et top:50px → actualisez.

AVANT



APRÈS



À FAIRE **Comparez : relative déplace visuellement la fenêtre, mais sa place d’origine reste réservée.**

B2 — absolute : l’enfant utilise un parent comme repère

C’est le principe qui sera réutilisé pour l’overlay de l’exercice.

PARENT DÉJÀ PRÉSENT

```
.window-container {
    position: relative;
}
```

ENFANT DÉJÀ PRÉSENT

```
.window.absolute {
    position: absolute;
    top: 0;
    left: 0;
    z-index: 1;
}
```

AVANT



APRÈS



POURQUOI RELATIVE SUR LE PARENT ?

Ici, relative sert surtout à donner un repère de coordonnées à l’enfant absolute. L’enfant absolute sort du flux normal.

À FAIRE **Test rapide : remplacez absolute par static, actualisez, puis remettez absolute.**

B3 — fixed et sticky : ici, on ne réécrit rien

Le test se fait en faisant défiler la page.

FIXED — DÉJÀ ÉCRIT

```
.window.fixed {
    position: fixed;
    top: 0;
    left: 0;
}
```

STICKY — DÉJÀ ÉCRIT

```
.window.sticky {
    position: sticky;
    top: 0;
}
```

FIXED

Référence = fenêtre du navigateur. La fenêtre reste au même endroit pendant le scroll.

STICKY

Commence dans le flux normal, puis « colle » lorsqu'elle atteint le seuil top:0.

RÉSUMÉ POSITION

static = normal | relative = décalé mais place conservée | absolute = hors flux, parent positionné | fixed = viewport | sticky = normal puis collant

À FAIRE **Dans B_position.html : faites défiler la page et observez fixed puis sticky.**

C — Flexbox : ouvrir C_flex.html

Flexbox se met sur le parent pour organiser ses enfants directs.

HTML

```
<div class="window-container flex">
    <div class="window">1...</div>
    <div class="window">2...</div>
    <div class="window">3...</div>
    ...
</div>
```

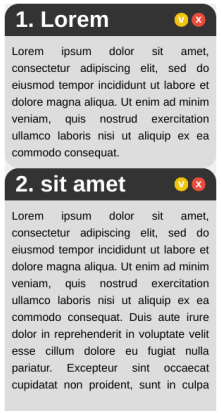
LIEN AVEC TD2

TD2 : on stylait surtout une boîte. Flexbox : le parent reçoit des règles pour organiser tout un groupe d'enfants.

STRUCTURE

.window-container.flex = PARENT / flex container
.window = ENFANTS DIRECTS / flex items

AVANT — bloc Flexbox désactivé pour comprendre le départ



À FAIRE **Pour le test, commentez temporairement le bloc .window-container.flex ; actualisez.**

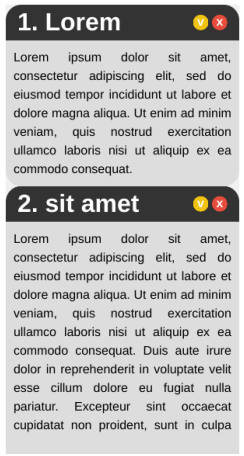
C1 — Activer Flexbox : display: flex

Cette propriété s’écrit sur le parent.

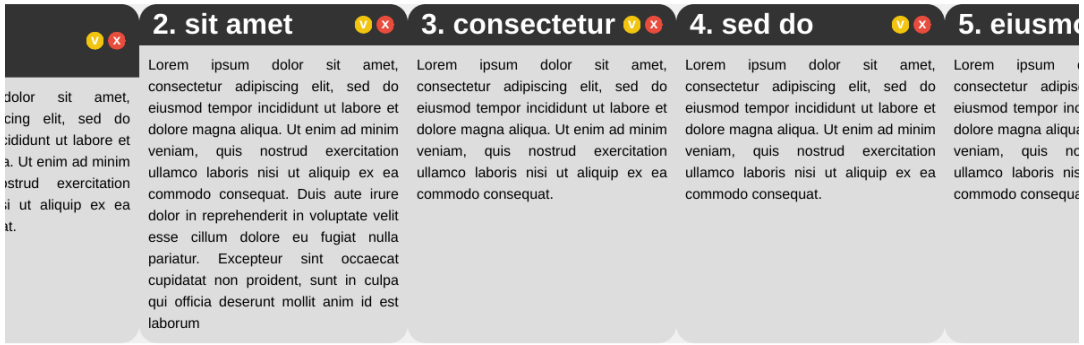
DANS .WINDOW-CONTAINER.FLEX

```
.window-container.flex {
    display: flex;
}
```

AVANT



APRÈS



CE QUI CHANGE

Le parent devient un flex container ; ses enfants directs deviennent des flex items et sont organisés par Flexbox.

À FAIRE **Laissez seulement display:flex actif dans le bloc Flexbox. Enregistrez et actualisez.**

C2 — flex-direction : row ou column

La direction définit l’axe principal du parent.

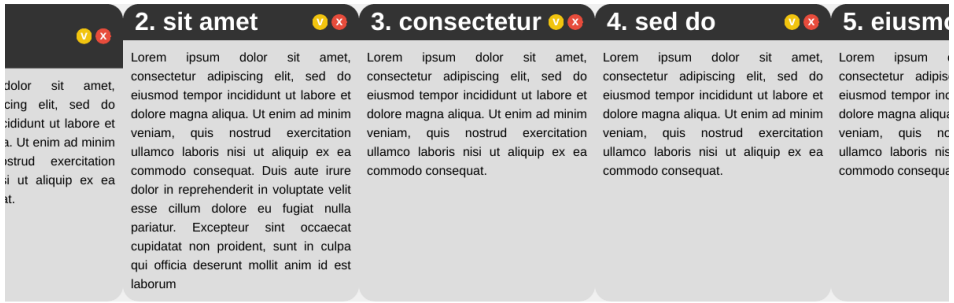
TEST 1

```
flex-direction: row;
```

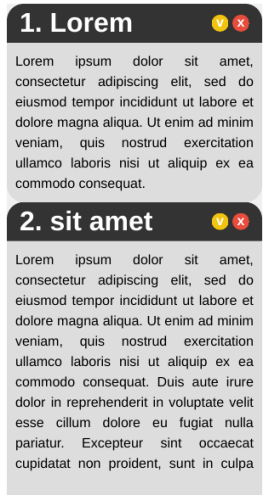
TEST 2

```
flex-direction: column;
```

ROW — horizontal



COLUMN — vertical



À RETENIR

row : axe principal horizontal. column : axe principal vertical.

À FAIRE **Testez row, puis remplacez par column. Revenez ensuite à row pour la suite.**

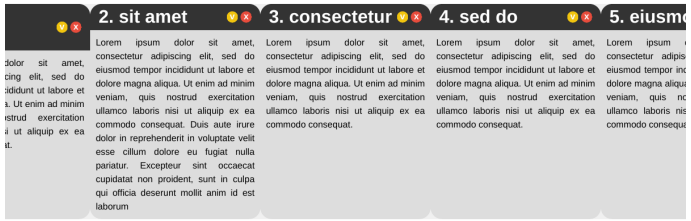
C3 — justify-content : déplacer le groupe sur l’axe principal

Gardez flex-direction: row pour voir clairement le déplacement horizontal.

TEST 1

```
justify-content: flex-start;
```

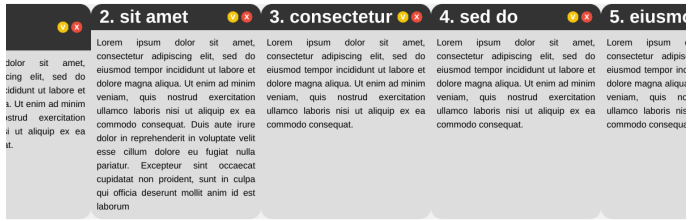
FLEX-START



TEST 2

```
justify-content: center;
```

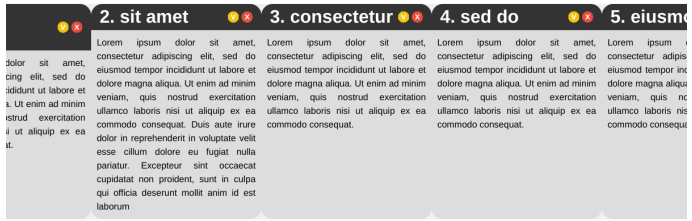
CENTER



TEST 3

```
justify-content: space-between;
```

SPACE-BETWEEN



À RETENIR

justify-content agit sur l’AXE PRINCIPAL. Avec row, cet axe est horizontal.

À FAIRE **Testez successivement flex-start, center puis space-between.**

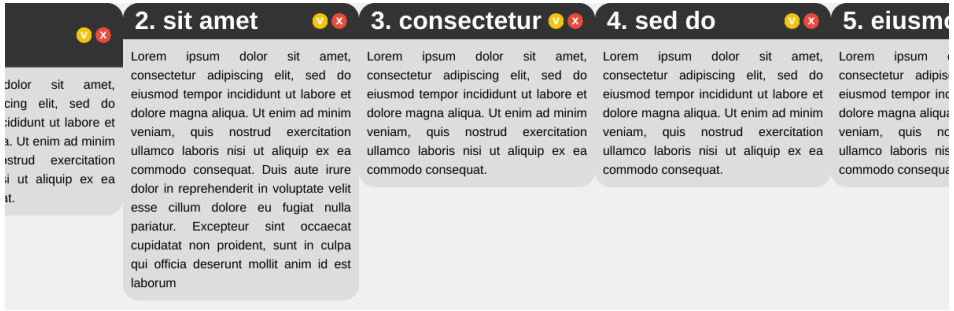
C4 — align-items : aligner sur l’axe croisé

Avec row : axe principal horizontal, axe croisé vertical.

TEST 1

```
align-items: flex-start;
```

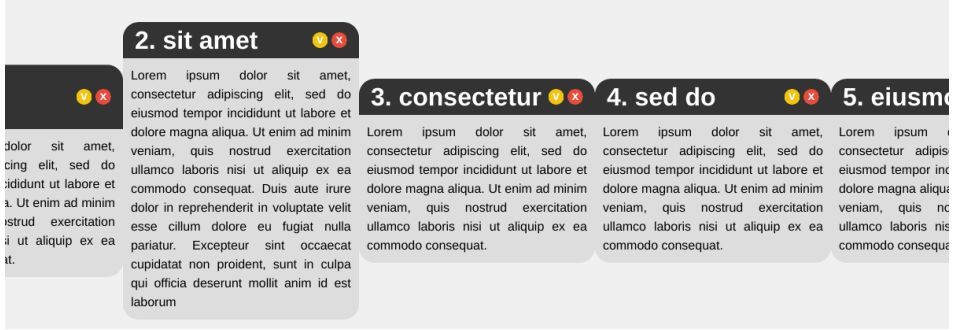
FLEX-START



TEST 2

```
align-items: center;
```

CENTER



DIFFÉRENCE

justify-content = axe principal. align-items = axe croisé. Les deux propriétés sont écrites sur le parent.

À FAIRE **Testez flex-start puis center. Observez surtout la fenêtre 2, plus haute que les autres.**

C5 — flex-wrap : que se passe-t-il quand la fenêtre devient étroite ?

Ici, il faut réellement réduire la largeur du navigateur.

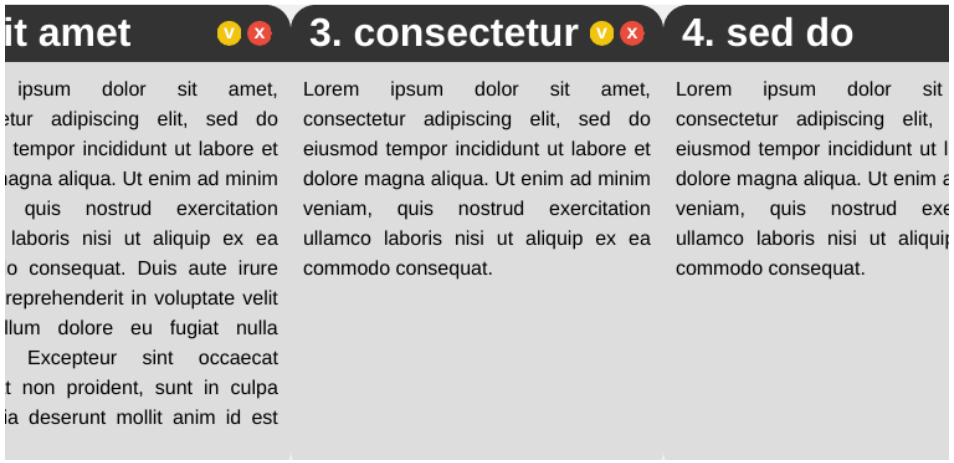
TEST 1

```
flex-wrap: nowrap;
```

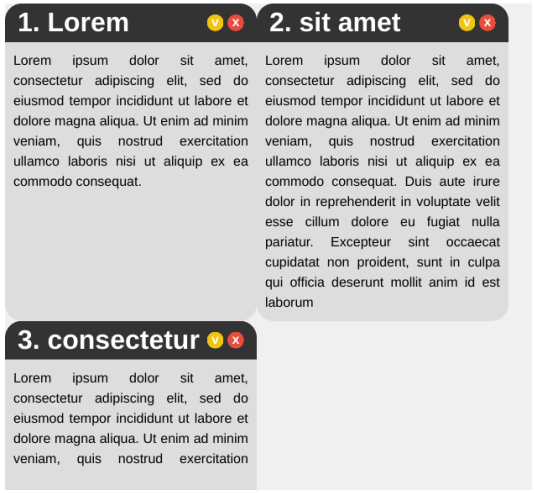
TEST 2

```
flex-wrap: wrap;
```

FENÊTRE ÉTROITE — NOWRAP



FENÊTRE ÉTROITE — WRAP



RÉSULTAT ATTENDU

Avec wrap, les flex items peuvent passer à la ligne quand la largeur disponible devient insuffisante.

À FAIRE **Testez nowrap puis wrap et réduisez la fenêtre du navigateur dans les deux cas.**

C6 — gap : gérer l’espace ENTRE les enfants

gap appartient au parent ; margin appartient à chaque élément.

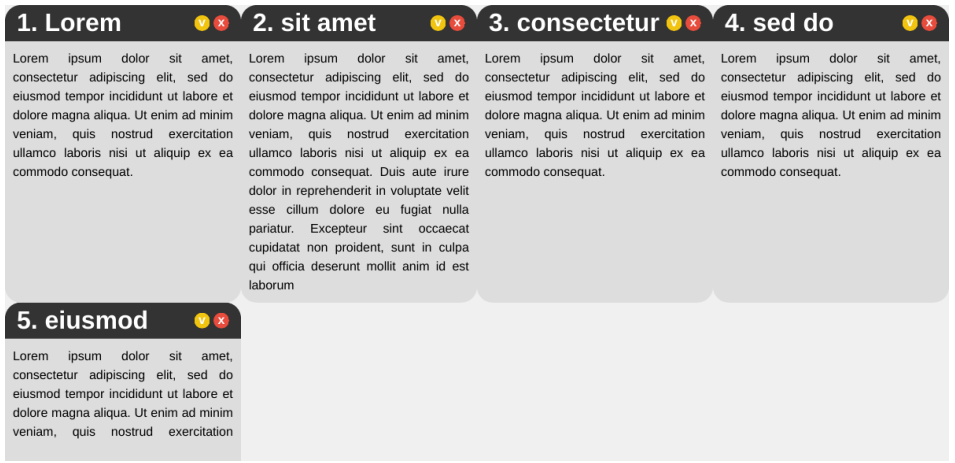
TEST 1

```
gap: 0;
```

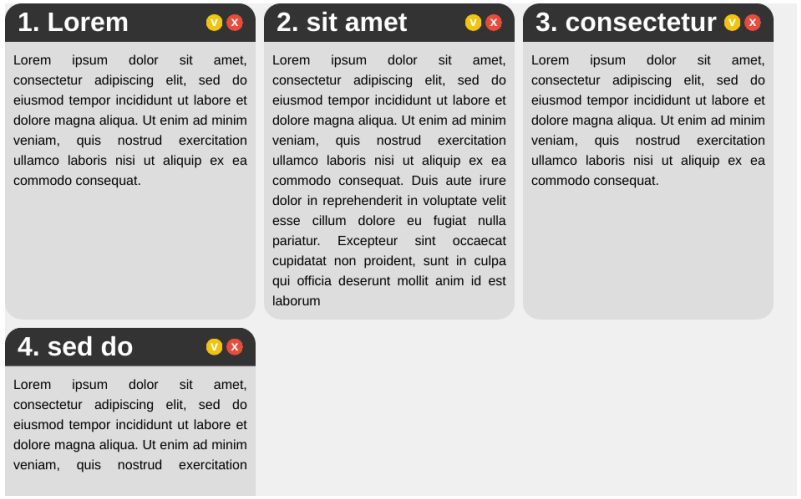
TEST 2

```
gap: 10px;
```

SANS GAP



GAP 10px



COMPARAISON AVEC TD2

margin : espace autour d’un élément. gap : espace géré par le parent entre ses flex items.

À FAIRE **Testez gap:0 puis gap:10px. Gardez ensuite la valeur du fichier fourni.**

Avant l’exercice : les 3 questions du TD3

SYNTHÈSE

Vous avez maintenant les outils nécessaires.

QUI ?

Sélecteurs

`.A.B, :not(), :nth-child(), :hover`

OÙ ?

position

`relative, absolute, fixed, sticky`

COMMENT ORGANISER ?

Flexbox

`display:flex, direction, justify, align, wrap, gap`

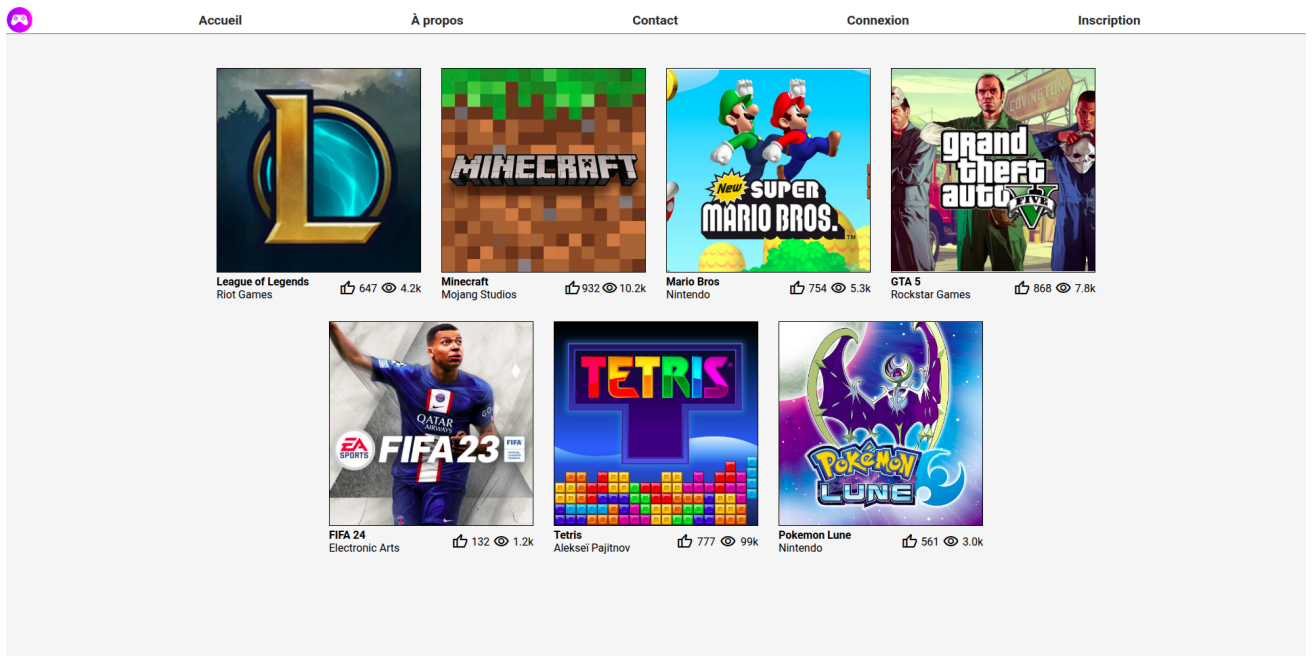
À FAIRE **Fermez les trois pages de démonstration. Ouvrez maintenant « TD3 — Exercice 1 ».**

Exercice autonome — objectif

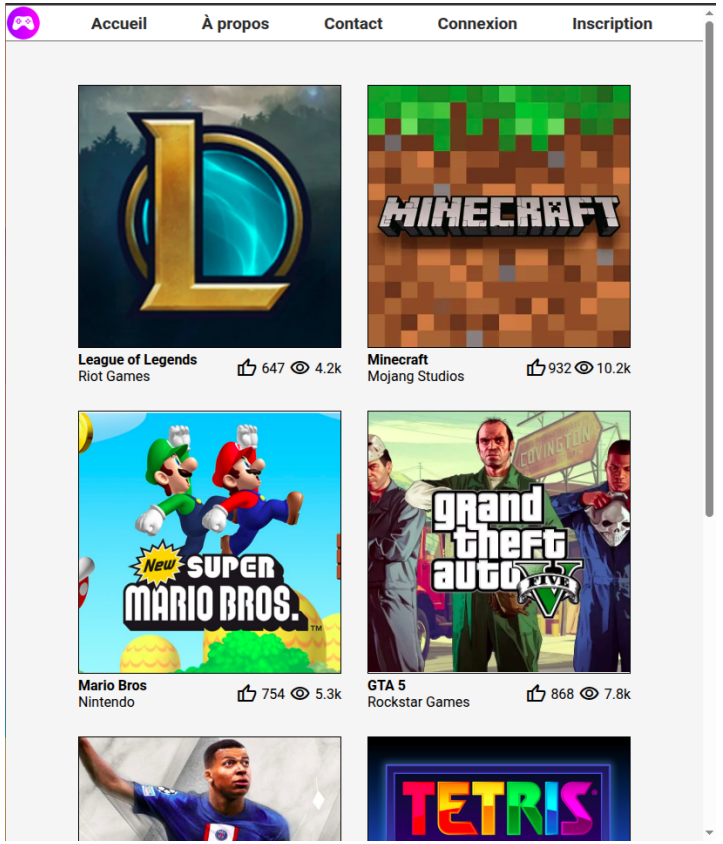
EXERCICE

Reproduire la page demandée en HTML + CSS.

CIBLE — écran large



ADAPTATION — fenêtre plus étroite



À FAIRE **Commencez par analyser la structure : header, conteneur des tuiles, tuile, overlay.**

Exercice autonome — avancez mission par mission

EXERCICE

Ne cherchez pas à tout résoudre en une seule fois.

1 STRUCTURE HTML

`HTML`

Créer le header, le conteneur des tuiles, une tuile et son overlay.

2 ORGANISER LES TUILES

`display:flex / wrap / gap`

Repérer le parent des tuiles et utiliser Flexbox.

3 HOVER DU HEADER

`:hover`

Les boutons changent de couleur au survol.

4 PLACER L'OVERLAY

`relative + absolute`

La tuile sert de repère ; l'overlay se place par-dessus.

5 CACHER / AFFICHER

`opacity: 0 / 1`

Overlay invisible au départ, visible au survol.

6 ANIMER

`transition: opacity 0.3s`

Rendre l'apparition progressive.

À FAIRE **Travaillez dans cet ordre. Si vous bloquez, revenez à la question : QUI ? OÙ ? COMMENT ORGANISER ?**